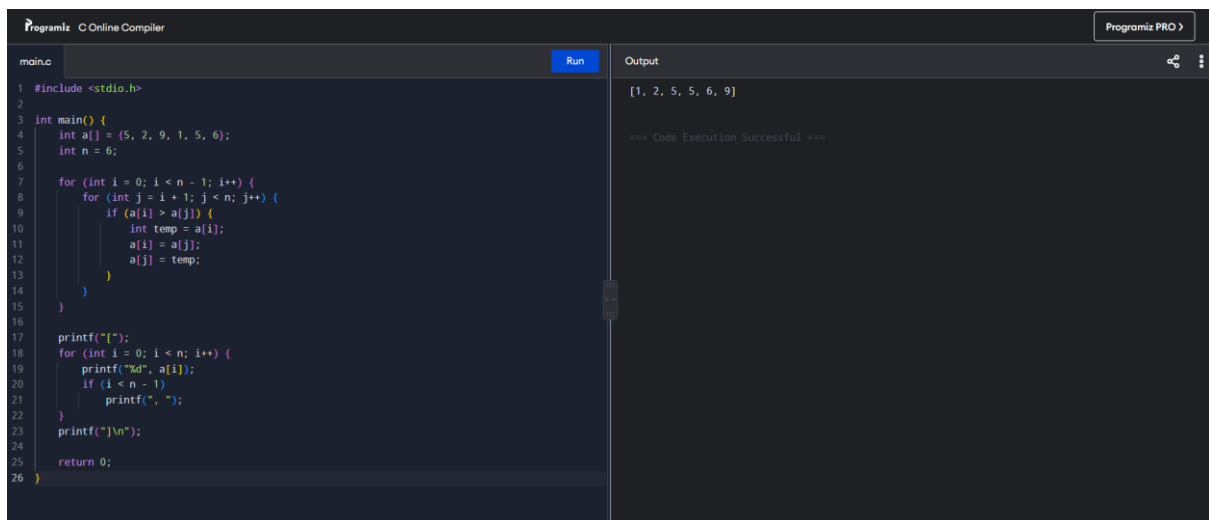


1. SORT A LIST



The screenshot shows a web-based C compiler interface. The left pane contains the source code for a program named 'main.c'. The code implements a bubble sort algorithm. It starts with an array 'a' containing the values {5, 2, 9, 1, 5, 6} and a size 'n' of 6. A nested loop structure compares adjacent elements and swaps them if they are in the wrong order. The right pane shows the output of the program, which is the sorted array [1, 2, 5, 5, 6, 9]. Below the output, it says '=== Code Execution Successful ==='.

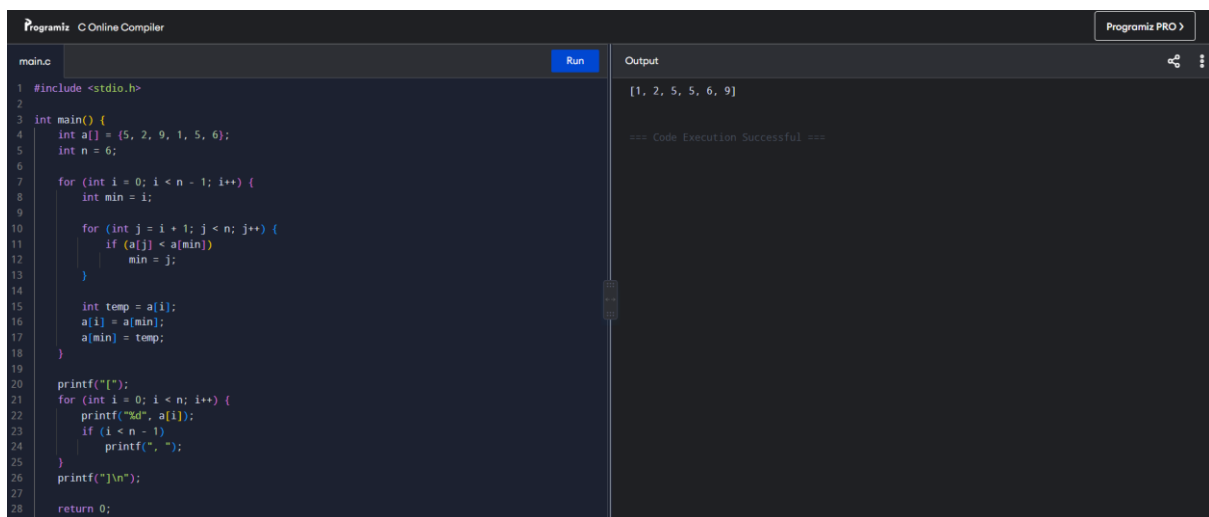
```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {5, 2, 9, 1, 5, 6};
5     int n = 6;
6
7     for (int i = 0; i < n - 1; i++) {
8         for (int j = i + 1; j < n; j++) {
9             if (a[i] > a[j]) {
10                 int temp = a[i];
11                 a[i] = a[j];
12                 a[j] = temp;
13             }
14         }
15     }
16
17     printf("[*]:\n");
18     for (int i = 0; i < n; i++) {
19         printf("%d", a[i]);
20         if (i < n - 1)
21             printf(", ");
22     }
23     printf("\n");
24
25     return 0;
26 }
```

Output

[1, 2, 5, 5, 6, 9]

=== Code Execution Successful ===

2. SELECTION SORT



The screenshot shows a web-based C compiler interface. The left pane contains the source code for a program named 'main.c'. The code implements a selection sort algorithm. It starts with an array 'a' containing the values {5, 2, 9, 1, 5, 6} and a size 'n' of 6. A nested loop structure finds the minimum element in the unsorted part of the array and swaps it with the first element of that part. The right pane shows the output of the program, which is the sorted array [1, 2, 5, 5, 6, 9]. Below the output, it says '=== Code Execution Successful ==='.

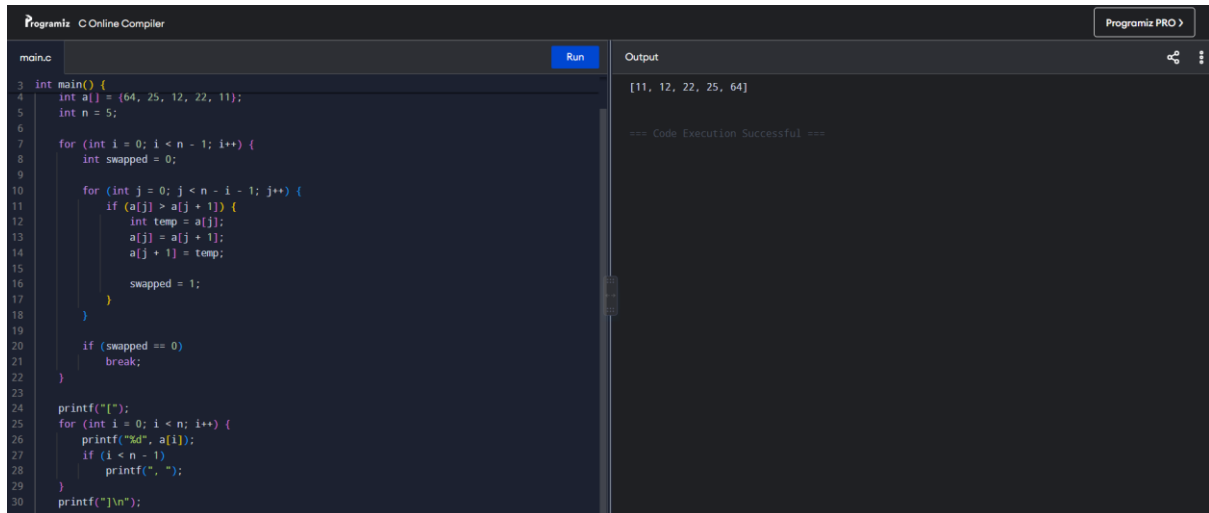
```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {5, 2, 9, 1, 5, 6};
5     int n = 6;
6
7     for (int i = 0; i < n - 1; i++) {
8         int min = i;
9
10        for (int j = i + 1; j < n; j++) {
11            if (a[j] < a[min])
12                min = j;
13        }
14
15        int temp = a[i];
16        a[i] = a[min];
17        a[min] = temp;
18    }
19
20    printf("[*]:\n");
21    for (int i = 0; i < n; i++) {
22        printf("%d", a[i]);
23        if (i < n - 1)
24            printf(", ");
25    }
26    printf("\n");
27
28    return 0;
29 }
```

Output

[1, 2, 5, 5, 6, 9]

=== Code Execution Successful ===

3. OPTIMIZED BUBBLE SORT

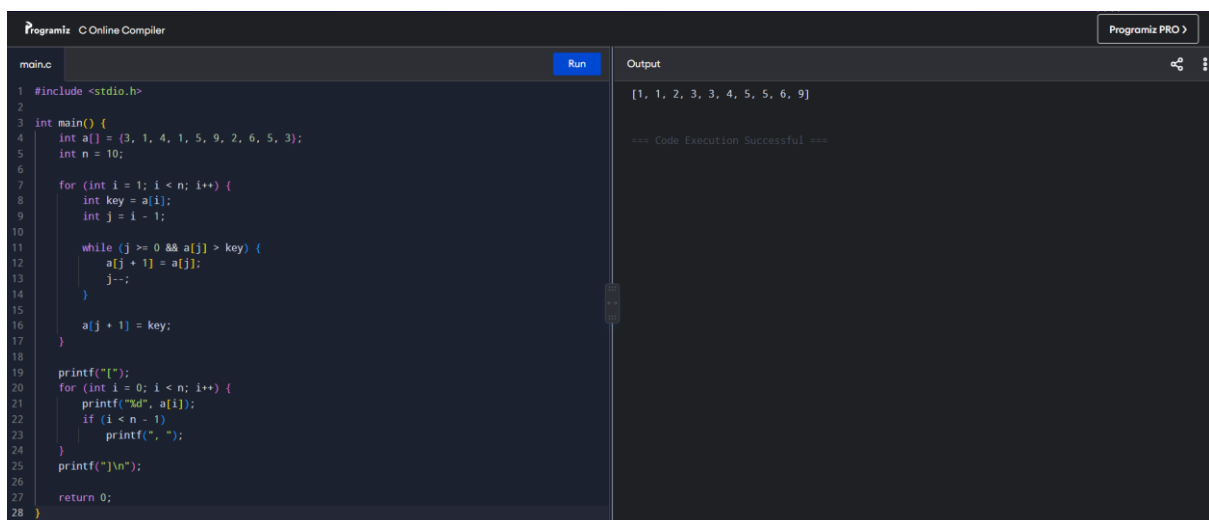


The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains the following C code for an optimized bubble sort:

```
1 main.c Run
2 int main() {
3     int a[] = {64, 25, 12, 22, 11};
4     int n = 5;
5
6     for (int i = 0; i < n - 1; i++) {
7         int swapped = 0;
8
9         for (int j = 0; j < n - i - 1; j++) {
10             if (a[j] > a[j + 1]) {
11                 int temp = a[j];
12                 a[j] = a[j + 1];
13                 a[j + 1] = temp;
14                 swapped = 1;
15             }
16         }
17
18         if (swapped == 0)
19             break;
20     }
21
22     printf("[");
23     for (int i = 0; i < n; i++) {
24         printf("%d", a[i]);
25         if (i < n - 1)
26             printf(", ");
27     }
28     printf("]\n");
29 }
```

The output panel on the right displays the sorted array: [11, 12, 22, 25, 64]. Below the output, it states "=== Code Execution Successful ===".

4. INSERTION SORT WITH DUPLICATE ELEMENTS

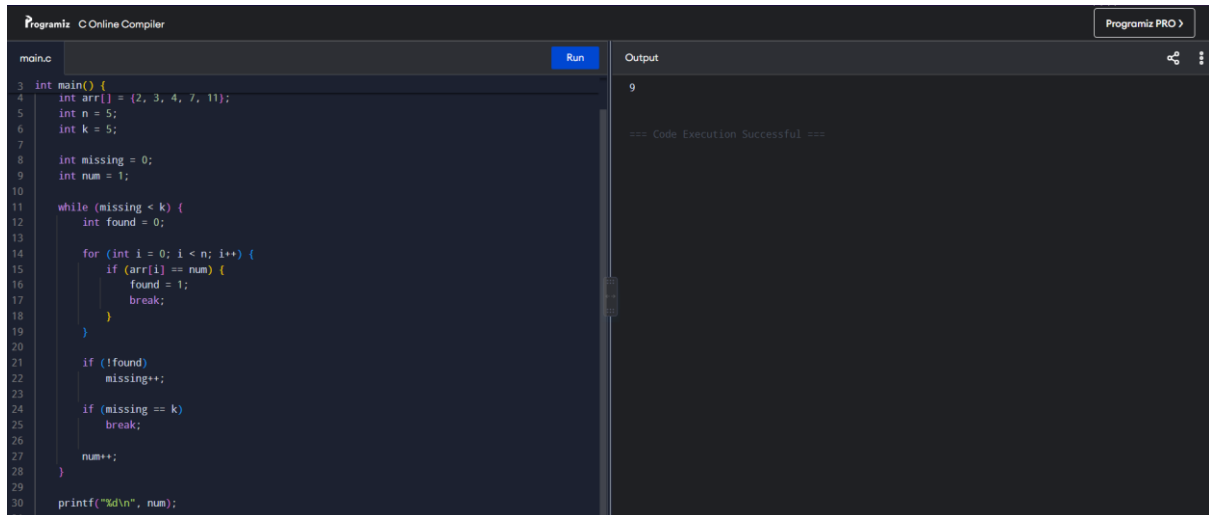


The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains the following C code for an insertion sort that handles duplicate elements:

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
5     int n = 10;
6
7     for (int i = 1; i < n; i++) {
8         int key = a[i];
9         int j = i - 1;
10
11         while (j >= 0 && a[j] > key) {
12             a[j + 1] = a[j];
13             j--;
14         }
15         a[j + 1] = key;
16     }
17
18     printf("[");
19     for (int i = 0; i < n; i++) {
20         printf("%d", a[i]);
21         if (i < n - 1)
22             printf(", ");
23     }
24     printf("]\n");
25
26     return 0;
27 }
```

The output panel on the right displays the sorted array: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]. Below the output, it states "=== Code Execution Successful ===".

5. KTH MISSING POSITIVE NUMBER



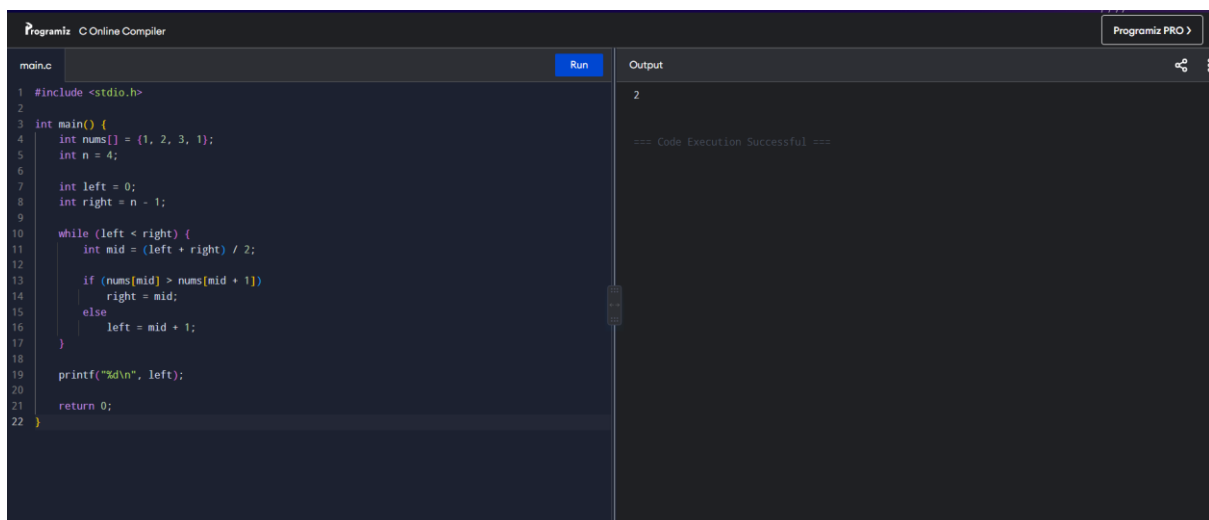
The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
1 //main.c
2 int main() {
3     int arr[] = {2, 3, 4, 7, 11};
4     int n = 5;
5     int k = 5;
6
7     int missing = 0;
8     int num = 1;
9
10    while (missing < k) {
11        int found = 0;
12
13        for (int i = 0; i < n; i++) {
14            if (arr[i] == num) {
15                found = 1;
16                break;
17            }
18        }
19
20        if (!found)
21            missing++;
22
23        if (missing == k)
24            break;
25
26        num++;
27    }
28
29    printf("%d\n", num);
30 }
```

The output on the right is:

```
9
=== Code Execution Successful ===
```

6. PEAK ELEMENT — $O(\log N)$



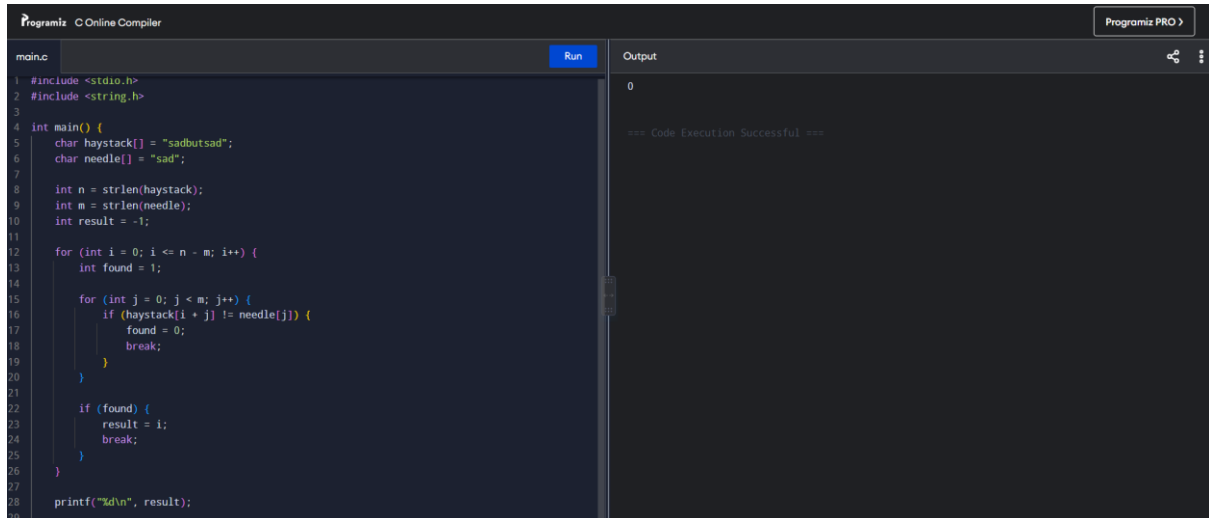
The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
1 //main.c
2 #include <stdio.h>
3
4 int main() {
5     int nums[] = {1, 2, 3, 1};
6     int n = 4;
7
8     int left = 0;
9     int right = n - 1;
10
11    while (left < right) {
12        int mid = (left + right) / 2;
13
14        if (nums[mid] > nums[mid + 1])
15            right = mid;
16        else
17            left = mid + 1;
18    }
19
20    printf("%d\n", left);
21
22    return 0;
23 }
```

The output on the right is:

```
2
=== Code Execution Successful ===
```

7. FIND FIRST OCCURRENCE OF A STRING



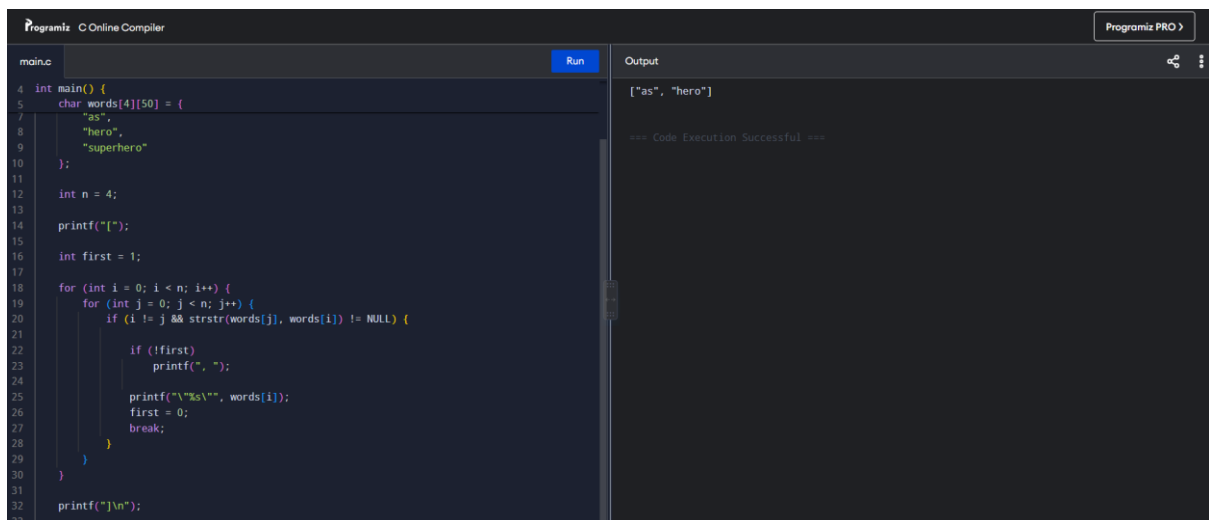
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program that finds the first occurrence of the string "sad" in the string "sadbutsad". The output window on the right shows the result "0".

```
main.c Run
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char haystack[] = "sadbutsad";
6     char needle[] = "sad";
7
8     int n = strlen(haystack);
9     int m = strlen(needle);
10    int result = -1;
11
12    for (int i = 0; i <= n - m; i++) {
13        int found = 1;
14
15        for (int j = 0; j < m; j++) {
16            if (haystack[i + j] != needle[j]) {
17                found = 0;
18                break;
19            }
20        }
21
22        if (found) {
23            result = i;
24            break;
25        }
26    }
27
28    printf("%d\n", result);
29 }
```

Output

```
0
=== Code Execution Successful ===
```

8. STRINGS THAT ARE SUBSTRINGS OF ANOTHER WORD



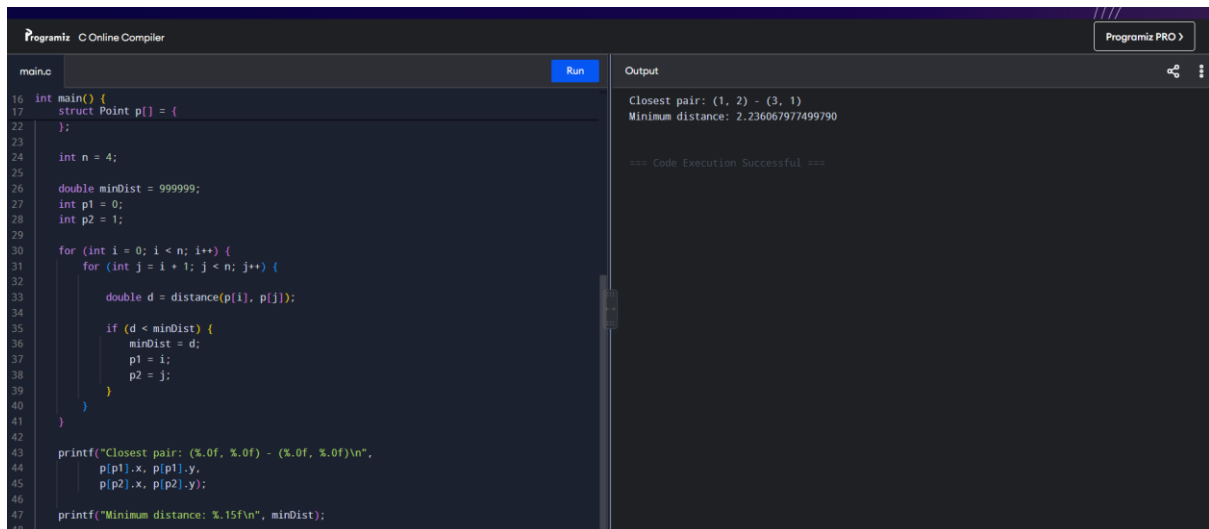
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program that finds all substrings of the word "as" in the array of words ["as", "hero", "superhero"]. The output window on the right shows the result ["as", "hero"].

```
main.c Run
4 int main() {
5     char words[4][50] = {
6         "as",
7         "hero",
8         "superhero"
9     };
10
11    int n = 4;
12
13    printf("[");
14
15    int first = 1;
16
17    for (int i = 0; i < n; i++) {
18        for (int j = 0; j < n; j++) {
19            if (i != j && strstr(words[j], words[i]) != NULL) {
20
21                if (!first)
22                    printf(", ");
23
24                printf("%s\\", words[i]);
25                first = 0;
26                break;
27            }
28        }
29    }
30
31    printf("]\n");
32 }
```

Output

```
["as", "hero"]
=== Code Execution Successful ===
```

9. CLOSEST PAIR OF POINTS



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

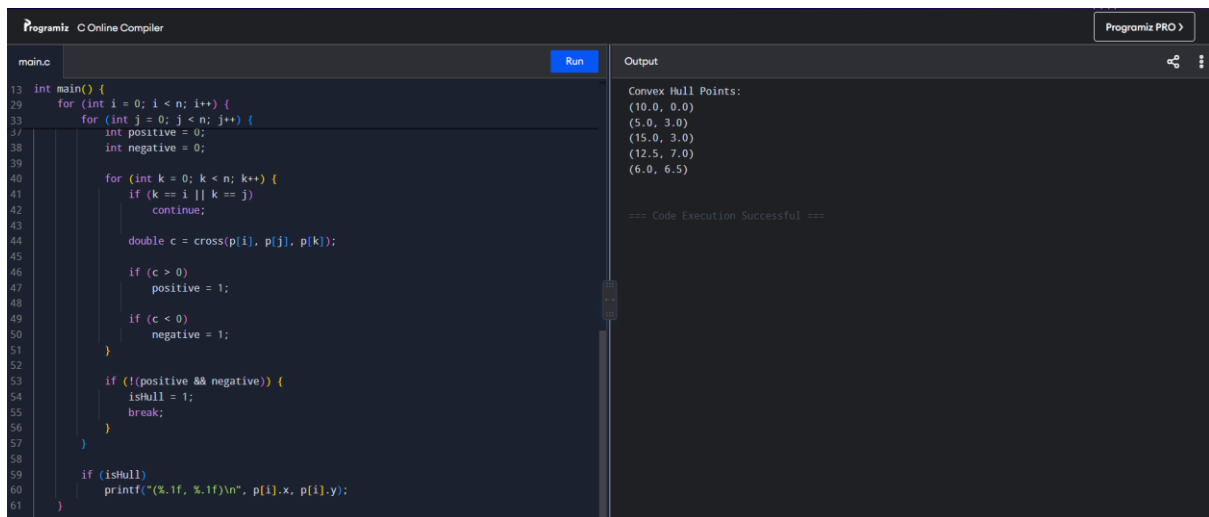
```
16 int main() {
17     struct Point p[] = {
22     };
23
24     int n = 4;
25
26     double minDist = 999999;
27     int p1 = 0;
28     int p2 = 1;
29
30     for (int i = 0; i < n; i++) {
31         for (int j = i + 1; j < n; j++) {
32
33             double d = distance(p[i], p[j]);
34
35             if (d < minDist) {
36                 minDist = d;
37                 p1 = i;
38                 p2 = j;
39             }
40         }
41     }
42
43     printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n",
44           p[p1].x, p[p1].y,
45           p[p2].x, p[p2].y);
46
47     printf("Minimum distance: %.15f\n", minDist);
48 }
```

The output on the right shows the result of the execution:

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.236067977499790

=== Code Execution Successful ===
```

10.CONVEX HULL — BRUTE FORCE



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

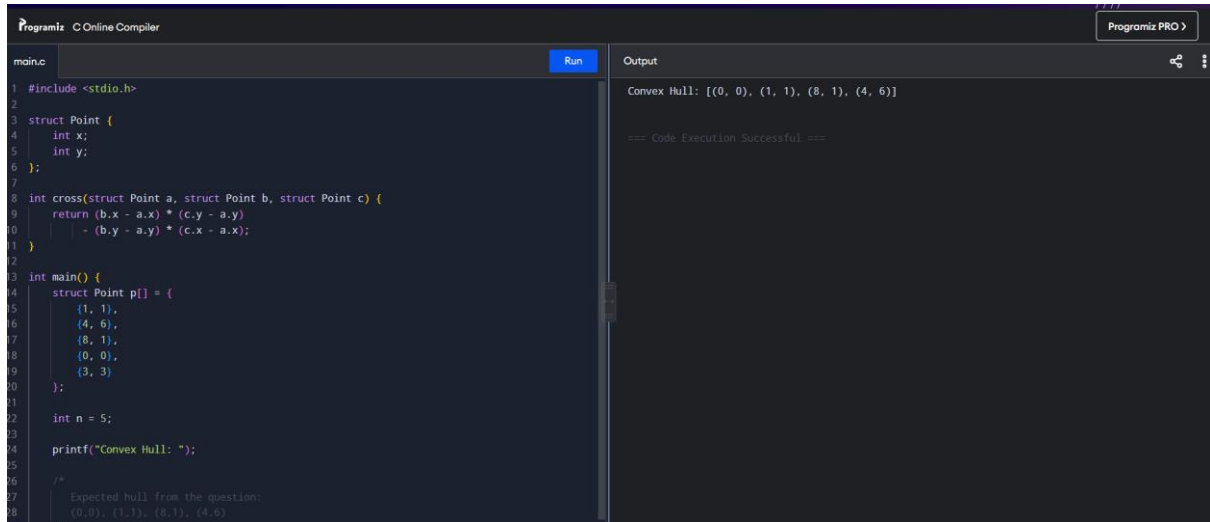
```
13 int main() {
29     for (int i = 0; i < n; i++) {
33         for (int j = 0; j < n; j++) {
37             int positive = 0;
38             int negative = 0;
39
40             for (int k = 0; k < n; k++) {
41                 if (k == i || k == j)
42                     continue;
43
44                 double c = cross(p[i], p[j], p[k]);
45
46                 if (c > 0)
47                     positive = 1;
48
49                 if (c < 0)
50                     negative = 1;
51             }
52
53             if (!(positive && negative)) {
54                 isHull = 1;
55                 break;
56             }
57         }
58
59         if (isHull)
60             printf("%.1f, %.1f\n", p[i].x, p[i].y);
61     }
62 }
```

The output on the right shows the result of the execution:

```
Convex Hull Points:
(10.0, 0.0)
(5.0, 3.0)
(15.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)

=== Code Execution Successful ===
```

11.CONVEX HULL OF 2D POINTS



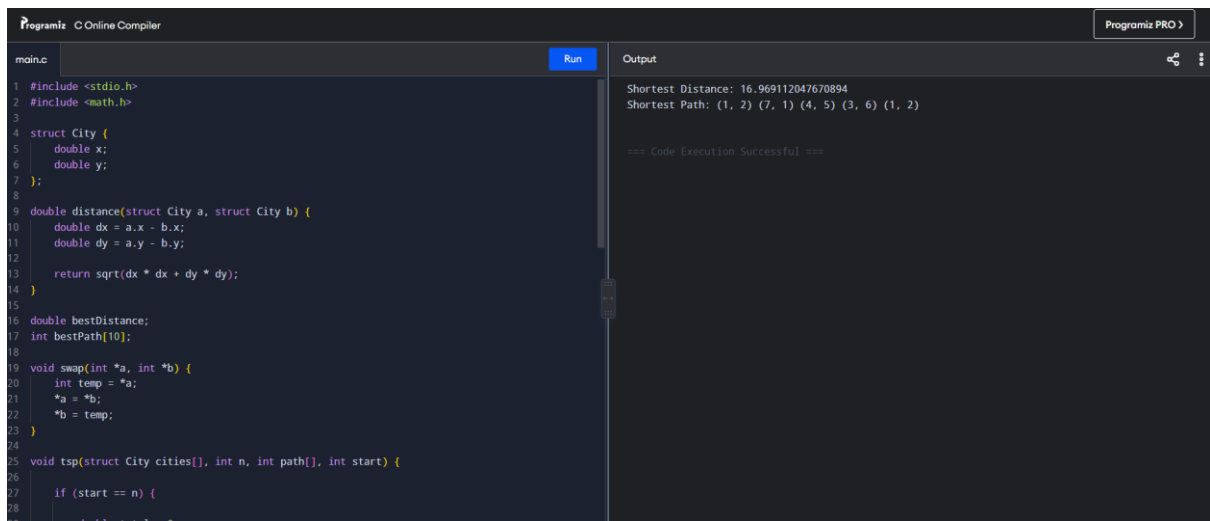
```
1 #include <stdio.h>
2
3 struct Point {
4     int x;
5     int y;
6 };
7
8 int cross(struct Point a, struct Point b, struct Point c) {
9     return (b.x - a.x) * (c.y - a.y)
10         - (b.y - a.y) * (c.x - a.x);
11 }
12
13 int main() {
14     struct Point p[] = {
15         {1, 1},
16         {4, 6},
17         {8, 1},
18         {0, 0},
19         {3, 3}
20     };
21
22     int n = 5;
23
24     printf("Convex Hull: ");
25
26     /*
27      Expected hull from the question:
28      (0,0), (1,1), (8,1), (4,6)
29     */
30 }
```

Output

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

=== Code Execution Successful ===

12. TRAVELLING SALESMAN PROBLEM — EXHAUSTIVE SEARCH



```
1 #include <stdio.h>
2 #include <math.h>
3
4 struct City {
5     double x;
6     double y;
7 };
8
9 double distance(struct City a, struct City b) {
10     double dx = a.x - b.x;
11     double dy = a.y - b.y;
12     return sqrt(dx * dx + dy * dy);
13 }
14
15 double bestDistance;
16 int bestPath[10];
17
18 void swap(int *a, int *b) {
19     int temp = *a;
20     *a = *b;
21     *b = temp;
22 }
23
24 void tsp(struct City cities[], int n, int path[], int start) {
25     if (start == n) {
26         double total = 0;
27         for (int i = 0; i < n; i++) {
28             total += distance(cities[path[i]], cities[path[i+1]]);
29         }
30         if (total < bestDistance) {
31             bestDistance = total;
32             for (int i = 0; i < n; i++) {
33                 bestPath[i] = path[i];
34             }
35         }
36     }
37     for (int i = start + 1; i < n; i++) {
38         path[start] = i;
39         tsp(cities, n, path, i);
40     }
41 }
```

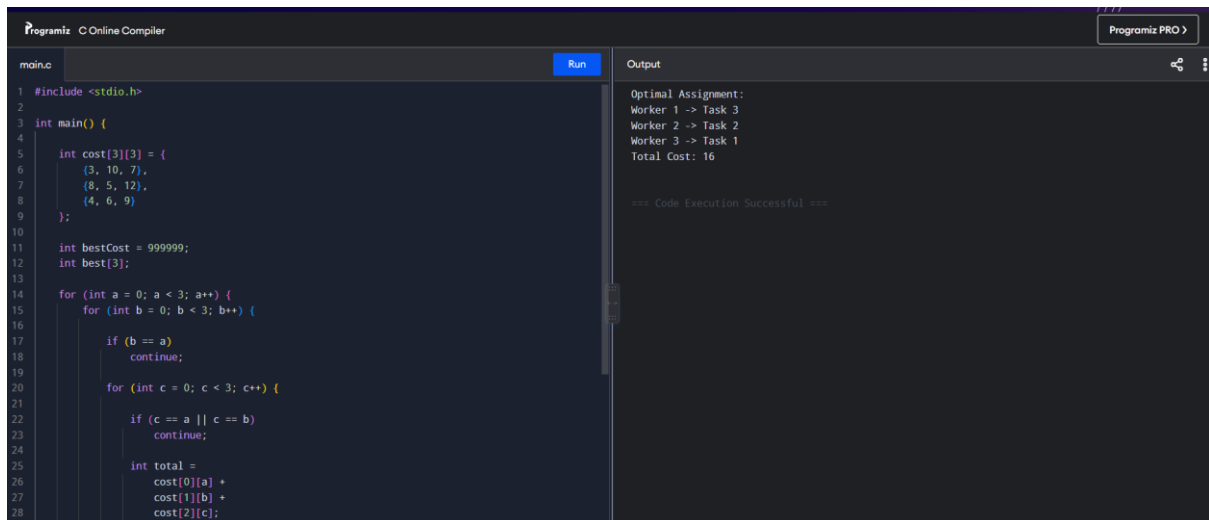
Output

Shortest Distance: 16.969112047670894

Shortest Path: (1, 2) (7, 1) (4, 5) (3, 6) (1, 2)

=== Code Execution Successful ===

13. ASSIGNMENT PROBLEM — EXHAUSTIVE SEARCH



The screenshot shows a C program in an online compiler. The code defines a 3x3 cost matrix and uses three nested loops to iterate through all possible assignments of 3 workers to 3 tasks. It checks for invalid assignments (where a worker is assigned multiple tasks) and calculates the total cost for each valid assignment. The optimal assignment is found and printed.

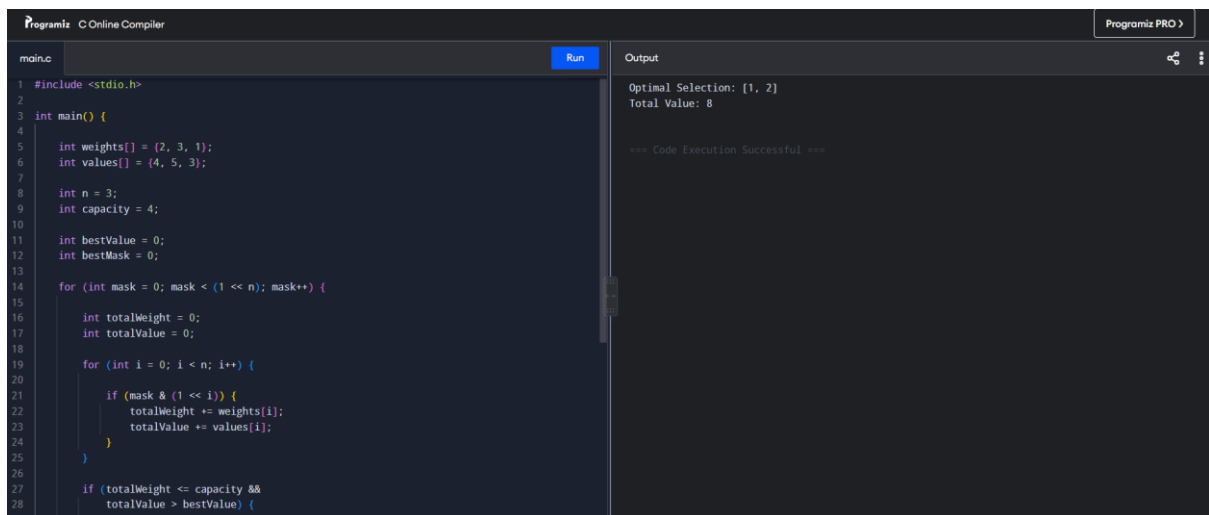
```
1 #include <stdio.h>
2
3 int main() {
4
5     int cost[3][3] = {
6         {3, 10, 7},
7         {8, 5, 12},
8         {4, 6, 9}
9     };
10
11     int bestCost = 999999;
12     int best[3];
13
14     for (int a = 0; a < 3; a++) {
15         for (int b = 0; b < 3; b++) {
16
17             if (b == a)
18                 continue;
19
20             for (int c = 0; c < 3; c++) {
21
22                 if (c == a || c == b)
23                     continue;
24
25                 int total =
26                     cost[0][a] +
27                     cost[1][b] +
28                     cost[2][c];
```

Output:

```
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16

== Code Execution Successful ==
```

14. 0-1 KNAPSACK — EXHAUSTIVE SEARCH



The screenshot shows a C program in an online compiler. The code defines an array of weights and values, the number of items, and the knapsack capacity. It uses a loop to iterate through all possible subsets of items (represented by bit masks) and calculates the total weight and value for each subset. The optimal selection is found and printed.

```
1 #include <stdio.h>
2
3 int main() {
4
5     int weights[] = {2, 3, 1};
6     int values[] = {4, 5, 3};
7
8     int n = 3;
9     int capacity = 4;
10
11     int bestValue = 0;
12     int bestMask = 0;
13
14     for (int mask = 0; mask < (1 << n); mask++) {
15
16         int totalWeight = 0;
17         int totalValue = 0;
18
19         for (int i = 0; i < n; i++) {
20
21             if (mask & (1 << i)) {
22                 totalWeight += weights[i];
23                 totalValue += values[i];
24             }
25         }
26
27         if (totalWeight <= capacity &&
28             totalValue > bestValue) {
```

Output:

```
Optimal Selection: {1, 2}
Total Value: 8

== Code Execution Successful ==
```